

ENTERPRISE DATA MANAGEMENT

พิชญ์ วัชรโชติเสถียร

พิชญ์ วัชรโชติเสถียร

SQL (Structured Query Language)

- SQL was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce after learning about the relational model from Edgar F. Codd in the early 1970s
 - This version, initially called SEQUEL (Structured English Query Language)
 - SQL is the standard language for Relational Database System
- SQL is a computer language for storing, manipulating and retrieving data stored in a relational database
- All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language
- These RDMS are using different dialects, such as
 - MS SQL Server using T-SQL,
 - Oracle using PL/SQL,
 - MS Access version of SQL is called JET SQL (native format) etc.

SQL Tutorials

- W3schools: <https://www.w3schools.com/sql/>
- SQLZoo: <http://sqlzoo.net/>
- CodeCademy: https://www.codecademy.com/learn/learn-sql?utm_source=pepperjam&utm_medium=affiliate&utm_term=214453&clickId=4083674639&pj_creativeid=8-12462&pj_publisherid=214453
- Khan Academy: <https://www.khanacademy.org/computing/computer-programming/sql>
- Free Code Camp: <https://www.youtube.com/watch?v=HXV3zeQKqGY>
- SQL Bolt: <https://sqlbolt.com>
- Datacamp

SQL Learning Tools

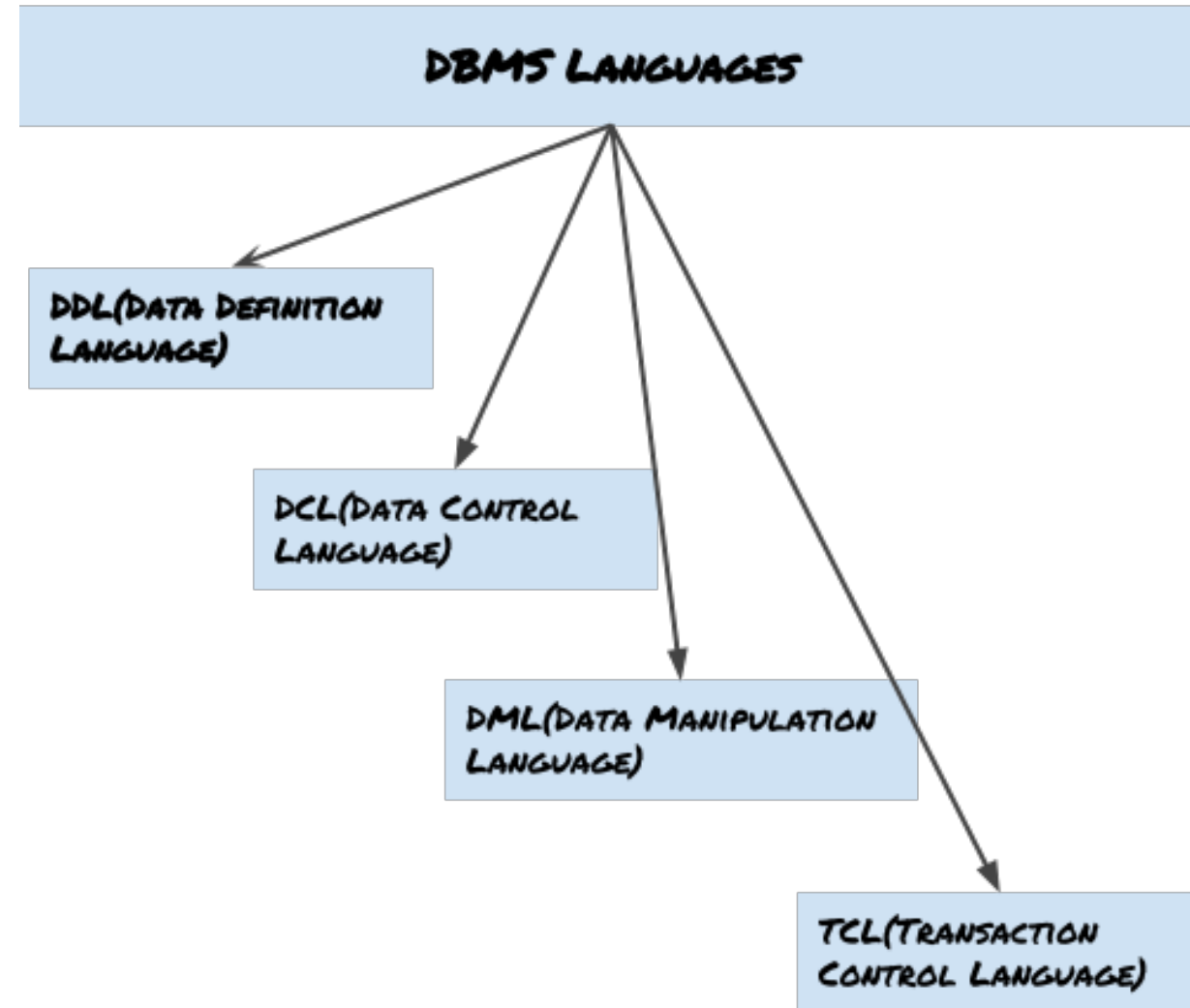
- Google Big Query: <https://console.cloud.google.com/bigquery>
- SQLite: <https://www.sqlite.org/download.html>



Types of DBMS Languages

Data Definition Language (DDL)

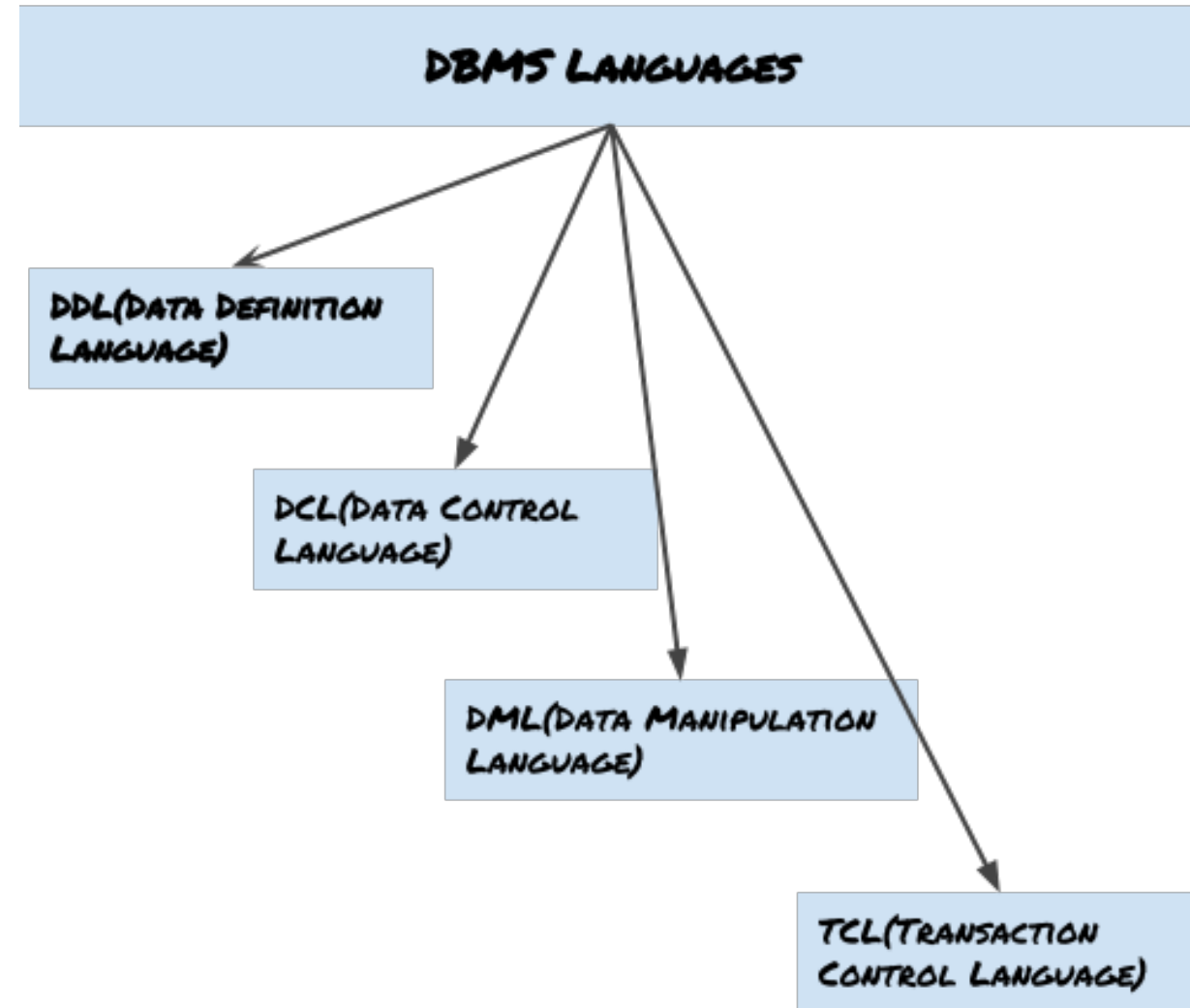
- DDL is used for specifying the database schema
- It is used for creating tables, schema, indexes, constraints etc. in database
- DDL operations:
 - To create the database instance – **CREATE**
 - To alter the structure of database – **ALTER**
 - To drop database instances – **DROP**
 - To delete tables in a database instance – **TRUNCATE**
 - To rename database instances – **RENAME**
 - To drop objects from database such as tables – **DROP**
 - To Comment – **Comment**



Types of DBMS Languages

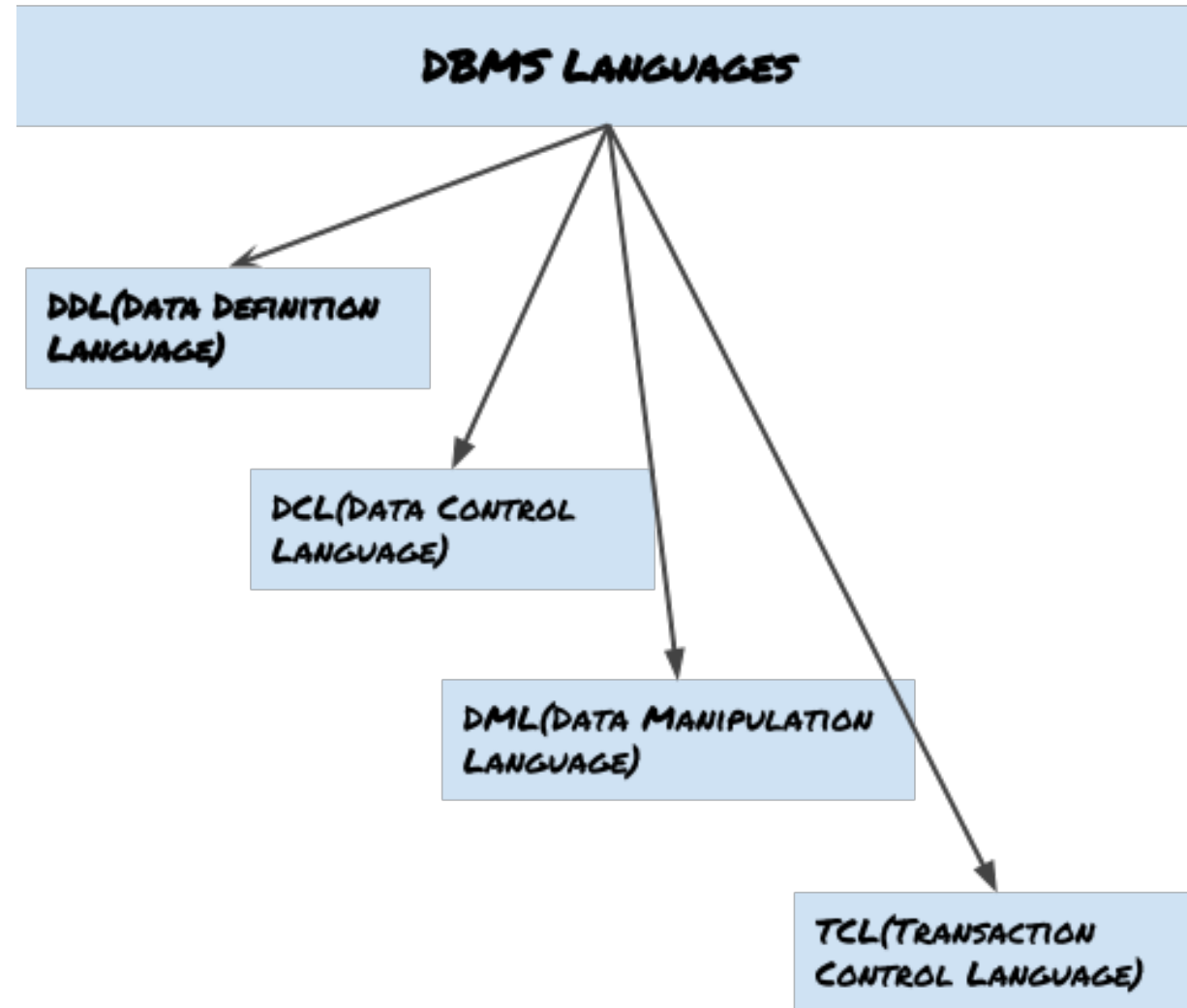
Data Manipulation Language (DML)

- DML is used for accessing and manipulating data in a database
- DML operations:
 - To read records from table(s) – SELECT
 - To insert record(s) into the table(s) – INSERT
 - Update the data in table(s) – UPDATE
 - Delete all the records from the table – DELETE



Types of DBMS Languages

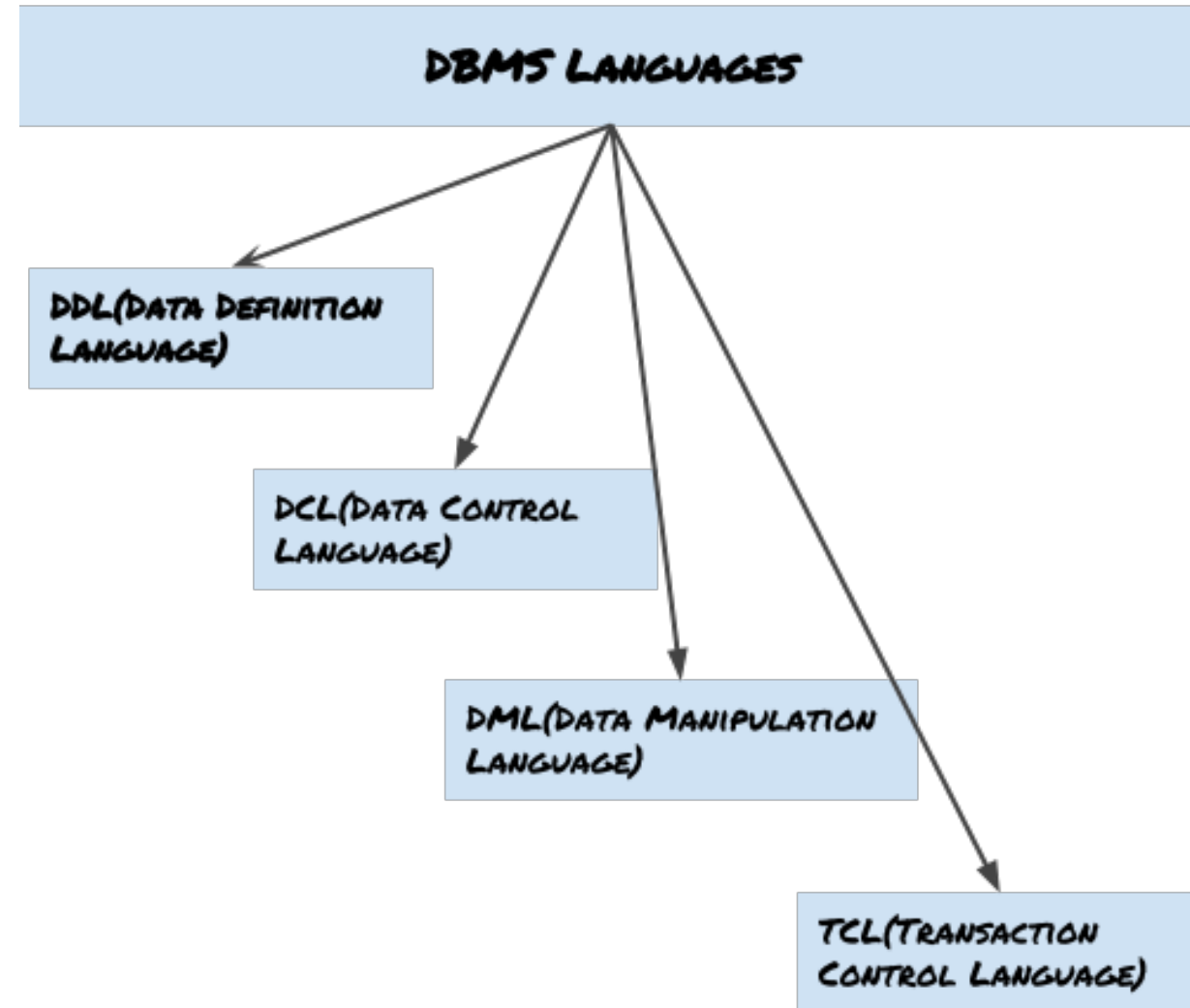
- DDL statements describe the structure of a database or schema, whereas DML statements allow altering data that already exists in the database
- We use the DDL commands for creating the database or schema, while DML commands are used to populate and manipulate the database
- DDL commands can affect the whole database or table, whereas DML statements only affect single or multiple rows based on the condition specified in a query



Types of DBMS Languages

Data Control language (DCL)

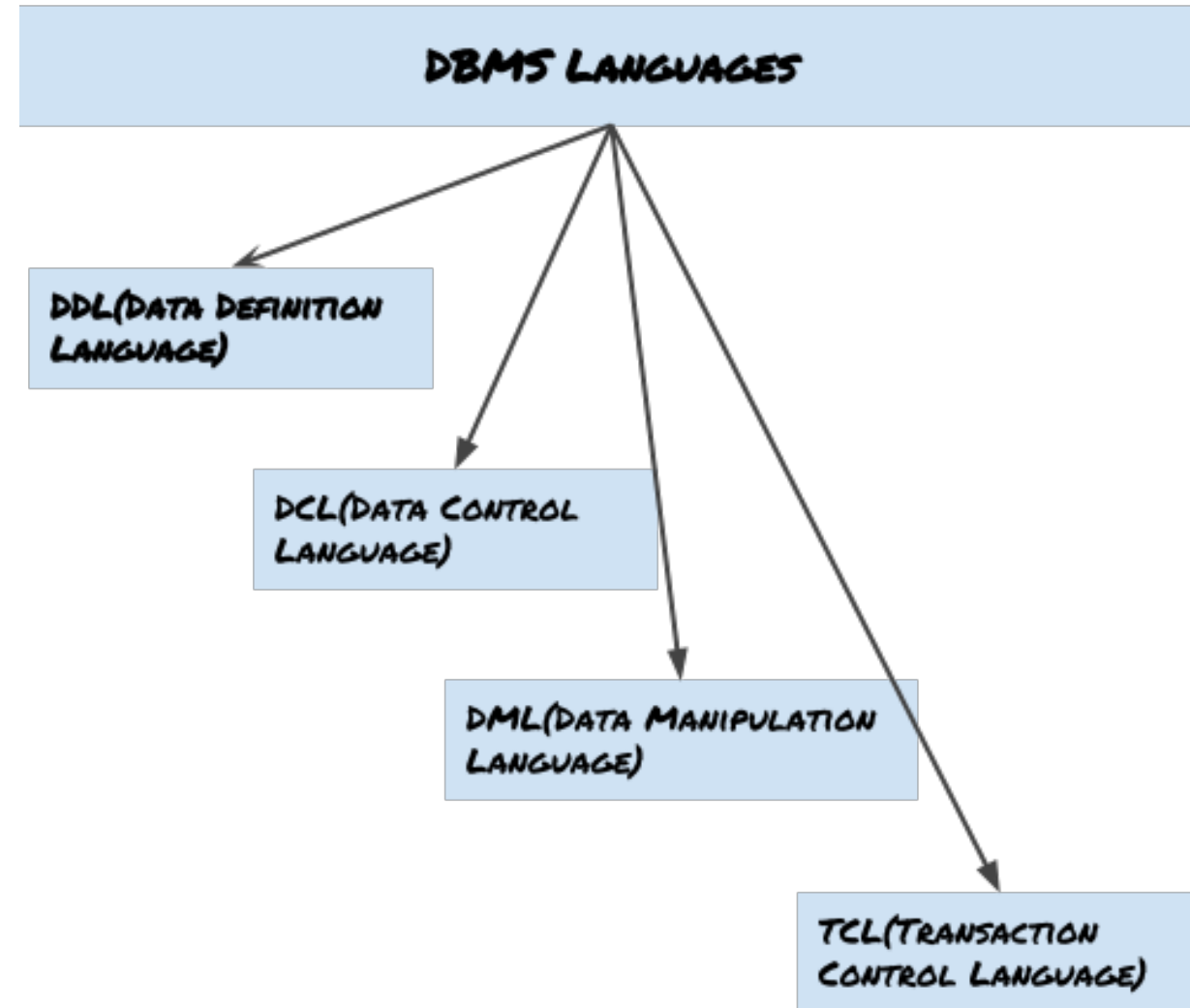
- DCL is used for granting and revoking user access on a database
 - To grant access to user – GRANT
 - To revoke access from user – REVOKE



Types of DBMS Languages

Transaction Control Language (TCL)

- The changes in the database that we made using DML commands are either performed or rolled back using TCL
 - To persist the changes made by DML commands in database – COMMIT
 - To rollback the changes made to the database – ROLLBACK



SQL (Structured Query Language)

- SQL was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce after learning about the relational model from Edgar F. Codd in the early 1970s
 - This version, initially called SEQUEL (Structured English Query Language)
 - SQL is the standard language for Relational Database System
- SQL is a computer language for storing, manipulating and retrieving data stored in a relational database
- All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language
- These RDMS are using different dialects, such as
 - MS SQL Server using T-SQL,
 - Oracle using PL/SQL,
 - MS Access version of SQL is called JET SQL (native format) etc.

SQL by Examples

Example Instance of Students Relation

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

- ❖ relation = rows (tuples) and columns (attributes).
- ❖ example: Cardinality = 3, degree = 5.
- ❖ Set not an ordered collection.

Basic SQL Commands

```
CREATE TABLE student(sNum INTEGER,sName VARCHAR (30)); -  
- creates table student with two columns
```

```
INSERT INTO student VALUES (1, 'Joe'); -- insert one row  
into the student table
```

```
SELECT * FROM student; -- select all rows from student  
table
```

```
DELETE FROM student; -- delete all rows in the student  
table
```

```
DROP TABLE student; -- drop student table
```

DDL --- Create

```
CREATE TABLE Students
    (sid: CHAR(20),
     name: CHAR(20),
     login: CHAR(10),
     age: INTEGER,
     gpa: REAL);
```

```
CREATE TABLE Enrolled
    (sid: CHAR(20),
     cid: CHAR(20),
     grade: CHAR(2));
```

DDL --- Altering Relations

Destroys the relation: The schema information *and* the tuples are deleted

```
DROP TABLE Students;
```

The schema of Students is altered by adding a new field (null value assigned)

```
ALTER TABLE Students  
    ADD COLUMN firstYear: integer;
```


DML --- Query Single Relation

Find all 18-year-old students

```
SELECT *  
FROM Students S  
WHERE S.age = 18;
```

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

DML --- Querying Multiple Relations

```
SELECT S.name, E.cid  
FROM Students S, Enrolled E  
WHERE S.sid=E.sid AND E.grade="A"
```

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

we get:

S.name	E.cid
Smith	Topology112

DML --- Adding and Deleting Tuples

- Insert a single tuple

```
INSERT INTO Students (sid, name, login, age, gpa)
VALUES ('53688', 'Smith', 'smith@ee', 18, 3.2);
```

- Delete tuples

```
DELETE FROM Students S
WHERE S.name = 'Smith';
```

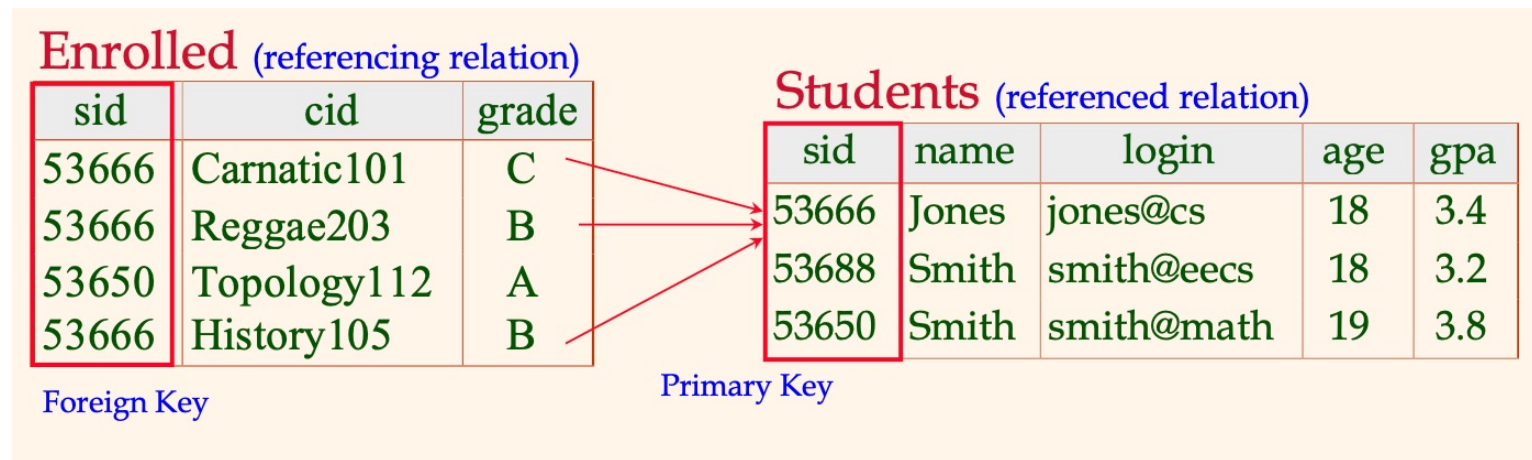
Primary and Candidate Keys

```
CREATE TABLE Student
    (sid CHAR(20),
     ssn CHAR(20),
     name CHAR(20),
     gpa INTEGER,
     PRIMARY KEY (sid),
     UNIQUE (ssn));
```

```
CREATE TABLE Enrolled
    (sid CHAR(20),
     cid CHAR(20),
     grade CHAR(2),
     PRIMARY KEY (sid,cid));
```

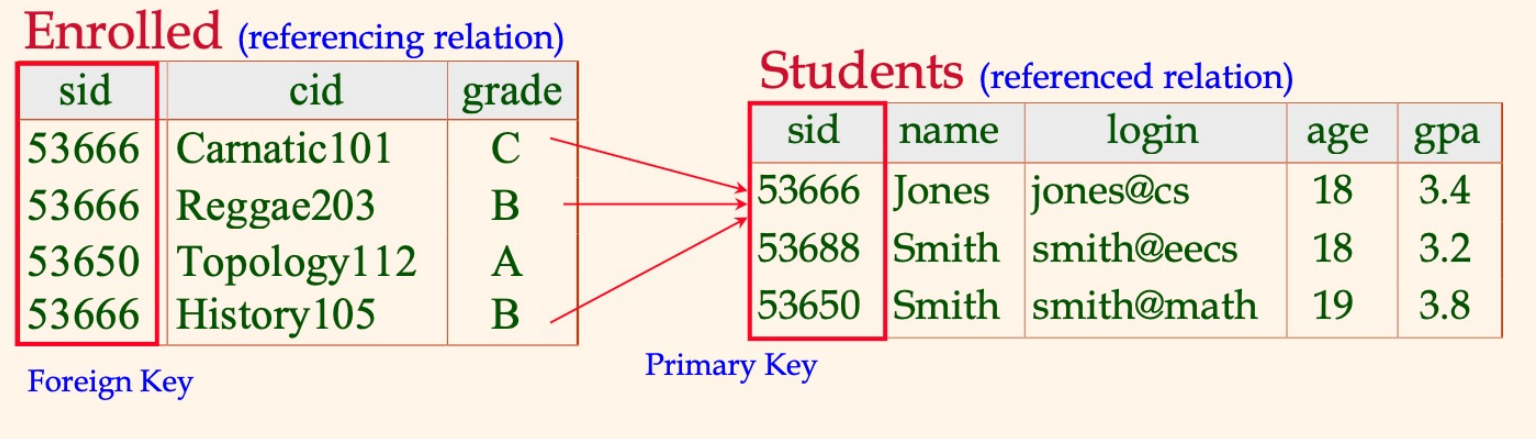
Foreign Keys

- Foreign key : Enrolled.sid → Students.sid
- If all foreign key constraints are enforced, then referential integrity is achieved, i.e., no dangling references



Foreign Keys

- Foreign key : Enrolled.sid → Students.sid
- If all foreign key constraints are enforced, then referential integrity is achieved, i.e., no dangling references



```
CREATE TABLE Enrolled
(sid CHAR(20),
cid CHAR(20),
grade CHAR(2),
PRIMARY KEY (sid,cid),
FOREIGN KEY (sid) REFERENCES Students )
```

SQL Syntax

<https://www.w3schools.com/sql/default.asp>

Create a database

```
CREATE DATABASE db_name;
```

- Need admin permission/ privilege before creating any database
- Check the list of databases with

```
SHOW DATABASES;
```


Drop a database

```
DROP DATABASE db_name;
```

- Be careful before dropping a database
- Deleting a database will result in loss of complete information stored in the database!

Backup a database

```
BACKUP DATABASE db_name  
TO DISK = 'filepath';
```

- Create a full back up of an existing SQL database
- Only back up the parts of the database that have changed since the last full database backup

```
BACKUP DATABASE db_name  
TO DISK = 'filepath'  
WITH DIFFERENTIAL;
```

Create a new table in a database

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    . . .  
);
```

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
    City varchar(255)  
);
```

PersonID	LastName	FirstName	Address	City

Data Types

Data type	Description
CHARACTER(n)	Character string จำกัดจำนวนตัวอักษรเท่ากับ n ตัว
VARCHAR(n) or CHARACTER VARYING(n)	Character string จำกัดจำนวนตัวอักษรไม่มากกว่า n ตัว
BINARY(n)	Binary string จำกัดจำนวนตัวอักษรเท่ากับ n bytes
VARBINARY(n) or BINARY VARYING(n)	Binary string จำกัดจำนวนตัวอักษรไม่มากกว่า n bytes
BOOLEAN	TRUE หรือ FALSE คือข้อมูลแบบตรรกศาสตร์
INTEGER(p)	ตัวเลขจำนวนเต็ม p หลัก
SMALLINT	ตัวเลขจำนวนเต็ม 5 หลัก
INTEGER	ตัวเลขจำนวนเต็ม 10 หลัก
BIGINT	ตัวเลขจำนวนเต็ม 19 หลัก
DECIMAL(p,s)	ตัวเลขที่มีจำนวนทั้งหมด p หลัก เป็นทศนิยม s หลัก และ จำนวนเต็ม p-s หลัก
NUMERIC(p,s)	ตัวเลขที่มีจำนวนทั้งหมด p หลัก เป็นทศนิยม s หลัก และ จำนวนเต็ม p-s หลัก
FLOAT(p)	ตัวเลขค่าประมาณรูปทศนิยมจำนวน p หลัก
REAL	ตัวเลขค่าประมาณรูปทศนิยมจำนวน 7 หลัก
FLOAT	ตัวเลขค่าประมาณรูปทศนิยมจำนวน 16 หลัก
DOUBLE PRECISION	ตัวเลขค่าประมาณรูปทศนิยมจำนวน 16 หลัก
DATE	ข้อมูล วัน,เดือน,ปี
TIME	ข้อมูล ชั่วโมง นาที วินาที
TIMESTAMP	ข้อมูล วัน เดือน ปี ชั่วโมง นาที วินาที
INTERVAL	ข้อมูลความต่างในทางเวลา
ARRAY	ชุดของข้อมูลที่มีการจัดเรียง
MULTISET	ชุดของข้อมูลที่ไม่มีการจัดเรียง อาจจะมีข้อมูลซ้ำได้
XML	ข้อมูลชนิด XML

Insert new records in a table

- Specify both the column names and the values to be inserted

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

- Add values for all the columns of the table

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

Insert new records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

Insert new records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	null	null	Stavanger	null	Norway

NULL Value

- A NULL value is different from a zero value or a field that contains spaces
- A field with a NULL value is one that has been left blank during record creation
- Test for NULL Values

```
SELECT column_names
FROM table_name
WHERE column_name IS <NOT> NULL;
```


SQL Operations

Operator	Operation	Description
=	Equal to	Checks if both operands have equal value, if yes, then returns TRUE
>	Greater than	Checks if the value of the left-hand operand is greater than the right-hand operand or not
<	Less than	Returns TRUE if the value of the left-hand operand is less than the value of the right-hand operand
>=	Greater than or equal to	It checks if the value of the left-hand operand is greater than or equal to the value of the right-hand operand, if yes, then returns TRUE
<=	Less than or equal to	Examines if the value of the left-hand operator is less than or equal to the right-hand operand

Operator	Operation	Description
<> or !=	Not equal to	Checks if values on either side of the operator are equal or not. Returns TRUE if values are not equal
!>	Not greater than	Used to check if the left-hand operator's value is not greater than or equal to the right-hand operator's value
!<	Not less than	Used to check if the left-hand operator's value is not less than or equal to the right-hand operator's value

Drop an existing table in a database

```
DROP TABLE table_name;
```

- Be careful before dropping a table, deleting a table will result in loss of complete information stored in the table!

```
TRUNCATE TABLE table_name;
```

- Delete the data inside a table, but not the table itself

Alter an existing table in a database

```
ALTER TABLE table_name  
ADD column_name datatype;
```

- Add a column in a table

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

- Delete a column in a table

```
ALTER TABLE table_name  
ALTER/MODIFY COLUMN column_name datatype;
```

- Change the data type of a column in a table (SQLite does not support)

SQL Constraints

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    ....  
);
```

- Constraints can be column level or table level
- If there is any violation between the constraint and the data action, the action is aborted
- [NOT NULL](#) - Ensures that a column cannot have a NULL value
- [UNIQUE](#) - Ensures that all values in a column are different
- [PRIMARY KEY](#) - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- [FOREIGN KEY](#) - Prevents actions that would destroy links between tables
- [CHECK](#) - Ensures that the values in a column satisfies a specific condition
- [DEFAULT](#) - Sets a default value for a column if no value is specified
- [CREATE INDEX](#) - Used to create and retrieve data from the database very quickly

SQL Constraints

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255) NOT NULL,  
    Age int  
);
```

```
ALTER TABLE Persons  
MODIFY Age int NOT NULL;
```

```
CREATE TABLE Persons (  
    ID int NOT NULL UNIQUE,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

A PRIMARY KEY constraint automatically has a UNIQUE constraint. However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

```
ALTER TABLE Persons  
ADD UNIQUE (ID);
```

```
ALTER TABLE Persons  
ADD CONSTRAINT UC_Person UNIQUE (ID,LastName);
```

```
ALTER TABLE Persons  
DROP CONSTRAINT UC_Person;
```

SQL Constraints

Primary Key

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    PRIMARY KEY (ID)  
);
```

```
ALTER TABLE Persons  
ADD PRIMARY KEY (ID);
```

```
ALTER TABLE Persons  
DROP PRIMARY KEY;
```

```
CREATE TABLE Persons (  
    ID int NOT NULL PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

```
ALTER TABLE Persons  
ADD CONSTRAINT PK_Person PRIMARY KEY (ID,LastName);
```

```
ALTER TABLE Persons  
DROP CONSTRAINT PK_Person;
```

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    CONSTRAINT PK_Person PRIMARY KEY (ID,LastName)  
);
```

ONE PRIMARY KEY (PK_Person) that
the VALUE of the primary key is
TWO COLUMNS (ID + LastName)

SQL Constraints

Foreign Key

- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table
- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

SQL Constraints

Foreign Key

- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table
- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables

PK

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

FK

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

SQL Constraints

Foreign Key

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID)  
    REFERENCES Persons(PersonID)  
);
```

```
CREATE TABLE Orders (  
    OrderID int NOT NULL PRIMARY KEY,  
    OrderNumber int NOT NULL,  
    PersonID int FOREIGN KEY REFERENCES Persons(PersonID)  
);
```

```
ALTER TABLE Orders  
ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

```
ALTER TABLE Orders  
DROP FOREIGN KEY FK_PersonOrder;
```

SQL Constraints

Check

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    CHECK (Age>=18)  
);
```

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255),  
    CONSTRAINT CHK_Person CHECK (Age>=18 AND City='Sandnes')  
);
```

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int CHECK (Age>=18)  
);
```

```
ALTER TABLE Persons  
ADD CHECK (Age>=18);
```

```
ALTER TABLE Persons  
DROP CHECK CHK_PersonAge;
```

SQL Constraints

Default

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Sandnes'  
);
```

```
ALTER TABLE Persons  
ALTER City SET DEFAULT 'Sandnes';
```

```
ALTER TABLE Persons  
ADD CONSTRAINT df_City  
DEFAULT 'Sandnes' FOR City;
```

```
ALTER TABLE Persons  
ALTER City DROP DEFAULT;
```

```
CREATE TABLE Orders (  
    ID int NOT NULL,  
    OrderNumber int NOT NULL,  
    OrderDate date DEFAULT GETDATE()  
);
```

current_timestamp for SQLite

```
ALTER TABLE Persons  
ALTER COLUMN City DROP DEFAULT;
```

SQL Constraints

Create index

- Updating a table with indexes takes more time than updating a table without (because the indexes also need an update)
- So, only create indexes on columns that will be frequently searched against

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

```
CREATE INDEX idx_lastname  
ON Persons (LastName);
```

```
DROP INDEX index_name ON table_name;
```

```
CREATE INDEX idx_pname  
ON Persons (LastName, FirstName);
```

```
ALTER TABLE table_name  
DROP INDEX index_name;
```

Select data from a database

```
SELECT column1, column2, ...  
FROM table_name;
```

```
SELECT * FROM table_name;
```

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

```
SELECT COUNT(DISTINCT Country) FROM Customers;
```

Note: COUNT(DISTINCT *column_name*) is not supported in MS Access on Firefox

```
SELECT Count(*) AS DistinctCountries  
FROM (SELECT DISTINCT Country FROM Customers);
```

Filter records

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

- The WHERE clause is used to filter records
- It is used to extract only those records that fulfill a specified condition

AND/OR/NOT Operators

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

```
SELECT * FROM Customers  
WHERE Country='Germany' AND (City='Berlin' OR City='München');
```

Sort the result

- Sort the records in ascending order by default

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

```
SELECT * FROM Customers  
ORDER BY Country DESC;
```

```
SELECT * FROM Customers  
ORDER BY Country ASC, CustomerName DESC;
```


Modify the existing records in a table

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

- Be careful when updating records in a table!
- Notice the **WHERE** clause in the **UPDATE** statement, the WHERE clause specifies which record(s) that should be updated
- If you omit the WHERE clause, all records in the table will be updated!

Modify the existing records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

Modify the existing records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

Modify the existing records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

```
UPDATE Customers  
SET ContactName='Juan';
```

Modify the existing records in a table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Juan	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Juan	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Juan	Berguvsvägen 8	Luleå	S-958 22	Sweden

```
UPDATE Customers  
SET ContactName='Juan';
```

Delete existing records in a table

```
DELETE FROM table_name WHERE condition;
```

- Be careful when deleting records in a table!
- Notice the WHERE clause in the DELETE statement, the WHERE clause specifies which record(s) should be deleted
- If you omit the WHERE clause, all records in the table will be deleted!

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

```
DELETE FROM Customers;
```

Specify the number of records to return

```
SELECT TOP number | percent column_name (s)  
FROM table_name  
WHERE condition;
```

- The SELECT TOP clause is useful on large tables with thousands of records
- Returning a large number of records can impact performance
- Not all database systems support the SELECT TOP clause
 - MySQL supports the LIMIT clause to select a limited number of records
 - Oracle uses FETCH FIRST *n* ROWS ONLY and ROWNUM.

```
SELECT TOP 3 * FROM Customers;    SELECT TOP 50 PERCENT * FROM Customers;    SELECT TOP 3 * FROM Customers  
WHERE Country='Germany';
```

Return the smallest / largest value of the selected column

```
SELECT MIN(column_name)  
FROM table_name  
WHERE condition;
```

```
SELECT MAX(column_name)  
FROM table_name  
WHERE condition;
```

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

SmallestPrice
2.5

COUNT(), AVG() and SUM() Functions

```
SELECT COUNT(column_name)  
FROM table name  
WHERE condition;
```

- Return the number of rows that matches a specified criterion

```
SELECT AVG(column_name)  
FROM table name  
WHERE condition;
```

- Return the average value of a numeric column

```
SELECT SUM(column_name)  
FROM table name  
WHERE condition;
```

- Return the total sum of a numeric column

COUNT(), AVG() and SUM() Functions

```
SELECT COUNT(column_name)  
FROM table name  
WHERE condition;
```

Find the number of products

- Return the number of rows that matches a specified criterion

```
SELECT COUNT(ProductID)  
FROM Products;
```

```
SELECT AVG(column_name)  
FROM table name  
WHERE condition;
```

- Return the average value of a numeric column

```
SELECT AVG(Price)  
FROM Products;
```

```
SELECT SUM(column_name)  
FROM table name  
WHERE condition;
```

- Return the total sum of a numeric column
- Note: NULL values are ignored

```
SELECT SUM(Quantity)  
FROM OrderDetails;
```

SQL Wildcard Characters

- A wildcard character is used to substitute one or more characters in a string
- Wildcard characters are used with the LIKE operator
- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column
- There are two wildcards often used in conjunction with the LIKE operator:
 - The percent sign (%) represents zero, one, or multiple characters
 - The underscore sign (_) represents one, single character
- Note: MS Access uses an asterisk (*) instead of the percent sign (%), and a question mark (?) instead of the underscore (_).

Wildcard Characters in MS Access

Symbol	Description	Example
*	Represents zero or more characters	bl* finds bl, black, blue, and blob
?	Represents a single character	h?t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
!	Represents any character not in the brackets	h[!oa]t finds hit, but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt
#	Represents any single numeric character	2#5 finds 205, 215, 225, 235, 245, 255, 265, 275, 285, and 295

Wildcard Characters in SQL Server

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
^	Represents any character not in the brackets	h[^oa]t finds hit, but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt

LIKE Operator

- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with "ber"

```
SELECT * FROM Customers  
WHERE City LIKE 'ber%';
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with "ber"

```
SELECT * FROM Customers
WHERE City LIKE 'ber%';
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City containing the pattern "es"

```
SELECT * FROM Customers  
WHERE City LIKE ...;
```


LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City containing the pattern "es"

```
SELECT * FROM Customers
WHERE City LIKE '%es%';
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with any character, followed by "ondon"

```
SELECT * FROM Customers
WHERE City LIKE ...;
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with any character, followed by "ondon"

```
SELECT * FROM Customers
WHERE City LIKE '_ondon';
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with "b", "s", or "p"

```
SELECT * FROM Customers
WHERE City LIKE 'b%' ;
```

LIKE Operator

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

- Select all customers with a City starting with "b", "s", or "p"

```
SELECT * FROM Customers
WHERE City LIKE '[bsp]%' ;
```

- Select all customers with a City starting with "a", "b", or "c"

```
SELECT * FROM Customers
WHERE City LIKE '[a-c]%' ;
```

- Select all customers with a City NOT starting with "b", "s", or "p"

```
SELECT * FROM Customers
WHERE City LIKE '[!bsp]%' ;

SELECT * FROM Customers
WHERE City NOT LIKE '[bsp]%' ;
```

BETWEEN Operator

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

- The BETWEEN operator selects values within a given range
 - The values can be numbers, text, or dates
- The BETWEEN operator is inclusive: begin and end values are included

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20;
```

```
SELECT * FROM Products
WHERE Price NOT BETWEEN 10 AND 20;
```

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20
AND CategoryID NOT IN (1,2,3);
```

```
SELECT * FROM Products
WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
ORDER BY ProductName;
```

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '1996-07-01' AND '1996-07-31';
```

SQL Aliases

- SQL aliases are used to give a table, or a column in a table, a temporary name
- Aliases are often used to make column names more readable
- An alias only exists for the duration of that query
- An alias is created with the AS keyword

```
SELECT column_name AS alias_name  
FROM table_name;
```

```
SELECT column_name(s)  
FROM table_name AS alias_name;
```

SQL Aliases

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10354	58	8	1996-11-14	3
10355	4	6	1996-11-15	1
10356	86	6	1996-11-18	2

```
SELECT CustomerName AS Customer, ContactName AS [Contact Person]
FROM Customers;
```

```
SELECT CustomerName, Address + ', ' + PostalCode + ' ' + City + ', ' + Country AS Address
FROM Customers;
```

```
SELECT Orders.OrderID, Orders.OrderDate, Customers.CustomerName
FROM Customers, Orders
WHERE Customers.CustomerName='Around the Horn' AND Customers.CustomerID=Orders.CustomerID;
```

```
SELECT o.OrderID, o.OrderDate, c.CustomerName
FROM Customers AS c, Orders AS o
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```


Comments

Single Line Comments

- Single line comments start with --

```
--Select all:
```

```
SELECT * FROM Customers;
```

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

- Any text between -- and the end of the line will be ignored (will not be executed)

Multi-line Comments

- Multi-line comments start with /* and end with */
- Any text between /* and */ will be ignored

```
/*Select all the columns  
of all the records  
in the Customers table:*/
```

```
SELECT * FROM Customers;
```

```
SELECT CustomerName, /*City,*/ Country FROM Customers;
```

Questions???

- To be continued...
 - SQL
 - JOIN
 - GROUP By
 - Exists
 - Any, All
 - Case
 - SELECT/INSERT INTO
 - Data Analysis & Dashboard